Lumina Moon Standard Optimizer Evaluation Methodology (LSO-Eval)

An Engineering Standard and Operational Procedure for Neural Network Optimization Algorithms

Silver Moon (`cloverxion`)

Lumina Moon (`cloverx-id`)

October 2026

Engineering Standard Specification — Version 1.1.0

Abstract

This manual defines the official **Lumina Moon Standard Optimizer Evaluation Methodology (LSO-Eval v1.1.0)**, an algorithm-agnostic verification and benchmarking standard for qualifying neural network optimizers and hardware acceleration backends. It establishes a repeatable, four-stage testing protocol to evaluate first-order, state-compressed, and quasi-second-order optimization algorithms alongside hardware execution backends (Rust AVX-512/AVX2 SIMD, Vulkan 1.3/1.4 Wave32 UMA, Triton AOT/JIT, and zero-copy NumPy/PyTorch frontends) before production deployment. LSO-Eval replaces ad-hoc trial-and-error with a structured four-stage filtering pipeline: (I) Formal Logic, Rational Padé Approximation, and Bitwise Precision Verification, (II) Synthetic Stress-Test Surfaces, Cross-Backend Numerical Parity, and State Serialization Integrity, (III) Short-Run Proxy Model Screening, and (IV) Large-Scale Pre-Training Qualification with Cryptographic Zero-Leakage Validation and Resource-Tiered Pareto Acceptance. Reference case studies throughout this specification illustrate the standard using the **LuminaV** optimizer (`v1.5.3`).

---

1 Scope, Principles, and the Staged Evaluation Pipeline

1.1 Objective and Operational Scope

The LSO-Eval standard governs the qualification of all first-order, state-compressed, and quasi-second-order optimization algorithms and hardware acceleration kernels developed or deployed within the Lumina Moon research and production infrastructure. Because LSO-Eval is an independent evaluation standard, its version lifecycle (`v1.1.0`) is decoupled from the version numbers of specific candidate algorithms. The objective is four-fold:

1. **Prevent Compute Wastage:** Filter out mathematically flawed, numerically unstable, or overly sensitive candidates at near-zero compute cost before allocating GPU clusters.
2. **Separate Implementation Bugs from Theory:** Isolate algebraic flaws, rational function approximation errors, low-precision stochastic rounding bugs, and hardware kernel overhead into distinct test phases.

3. **Guarantee Cross-Backend Consistency:** Enforce strict numerical parity across Scalar Reference Implementations, x86_64 AVX-512/AVX2 SIMD, Vulkan 1.3/1.4 UMA Wave32 compute shaders, Triton AOT/JIT GPU kernels, and NumPy/PyTorch bindings (`PyO3 abi3/abi3t`).
4. **Enforce Fair Multi-Objective Non-Regression:** Evaluate candidate optimizers across both validation quality and isolated hardware footprint (separating pure optimizer state memory from fixed model and activation baselines) while preventing regressions across releases.

1.2 Staged Filtering Protocol

The LSO-Eval pipeline is partitioned into two operational tiers across four sequential core stages plus modality and version-regression extensions (Table 1): **Stages I and II** serve as the mandatory *pre-release and continuous integration (CI) gate* executed on CPU for every software or kernel release, whereas **Stages III, IV, and V-Eval** serve as the *post-release empirical qualification and major algorithmic benchmark* executed on GPU hardware (or by independent researchers evaluating new optimization dynamics across language and vision modalities). Failure at any lower stage halts progression to higher-cost GPU stages immediately.

Table 1: The LSO-Eval Staged Evaluation Pipeline (v1.1.0)

Stage	Designation	Core Verification Objective	Cost Ceiling
Stage I	Math & Bitwise Check	Zero division-by-zero, unit velocity bounds, subnormal floor safety, [13/6] Padé tanh accuracy, zero stochastic rounding bias.	$\mathcal{O}$(ms)
Stage II	Synthetic & Parity	Ill-conditioned traversal ($\kappa \geq 1000$), saddle escape, λ^* balance, 1-Pass EMA fusion, cross-backend numerical parity, 6-slot scratch & checkpoint integrity.	$\mathcal{O}$(sec)
Stage III	Short-Run Proxy	Matrix update-to-weight ratio stability ($R_{t,l} \in [10^{-7}, 0.10]$), spike recovery ($F_2$), low-precision stall gating ($F_3 < 0.25$).	$\mathcal{O}$(min)
Stage IV	Large-Scale Run	Zero-leakage WAL validation, low-precision BF16/FP16 stability, isolated optimizer VRAM (VRAM_{opt}) vs. peak VRAM, kernel throughput, Two-Tiered Pareto held-out perplexity.	$\mathcal{O}$(hours)
V-Eval	Vision Benchmarks	Track V-A (from-scratch training across 5 vision models) and Track V-B (optional downstream detection & fine-tuning) across 6 BF16 vs. FP32-master optimizers.	$\mathcal{O}$(hours)
D-Eval	Version Regression	4-Gate comparative audit evaluating release candidate v_k against baseline predecessor v_{k-1}.	Cross-Stage

2 Stage I: Formal Logic & Symbolic Verification Protocol

Stage I requires formal verification of algebraic safety properties and bitwise precision bounds before running any tensor training loop. The candidate optimizer is expressed in symbolic real arithmetic and checked using automated theorem provers and statistical equivalence tests.

2.1 Standard SMT Safety Checklist (Z3 Solver)

The engineering team formulates the optimizer update velocity $u_t(\theta, g, m, v)$ and checks seven core safety categories (implemented via twelve negated sub-queries in the reference harness) using the Z3 Theorem Prover [1]. Each property is verified by proving that its **negation** is `UNSAT` (unsatisfiable):

Q1. Denominator Non-Singularity (Division-by-Zero): For any gradient $g \in \mathbb{R}$, momentum $m \in \mathbb{R}$, and instantaneous defect residual $d_t = g - m$, prove that the envelope denominator $\mathcal{E}$ (or variance divisor σ) strictly satisfies $\mathcal{E} > 0$ [2]. Negated query: $\mathcal{E} \leq 0$.

Q2. Velocity Regularity, Trust-Region Radii & Profile Boundedness: Prove that the update velocity and trust-region governors satisfy their design profile:

- *Profile A (Coordinate-Bounded / Normalized, e.g., LuminaV, Lion [3]):* Strictly bounded magnitude $|u_t| < 1.0$, positive coordinate trust radius $B_i^2 = r_{\text{bound}}^2(\theta_i^2 + 10^{-4}) > 0$, and radial trust lower bound $R = r_{\text{bound}} \max(\|\theta\|_2, 1.0) \geq r_{\text{bound}}$ [4]. Negated queries: $|u_t| \geq 1.0$, $B_i^2 \leq 0$, or $R < r_{\text{bound}}$.
- *Profile B (Variance-Adaptive, e.g., AdamW [5], AdaBelief [6], ADOPT [7]):* Clamped variance divisor strictly positive $\sigma_{\text{clamped}} = \max(c_1\sqrt{v_t} + c_2, 10^{-12}) > 0$ and non-divergent quotient as $g \to 0$. Negated query: $\sigma \leq 0 \vee \sigma_{\text{clamped}} \leq 0$.
- *Profile C (Matrix-Preconditioned / Subspace-Projected, e.g., Shampoo [8], GaLore [9]):* Spectral/operator norm boundedness $\|U_t\|_2 \leq C$ and non-singular preconditioning $\sigma_{\min}(P) > 0$.

Q3. Radial Scaling & Cautious Rescaling Bounds: For any rational radial scaling factor $S(r)$ (such as the Padé bound $(15 + r^2)/(15 + 6r^2)$ for $r > 0$ and 1.0 at $r = 0$), prove that $0 < S(r) \leq 1.0$ for all $r \geq 0$. For cautious alignment masking [10] with density floor $c_{\min} \in (0, 1]$, prove that the rescaling multiplier $\gamma_{\text{cautious}} = 1/\text{clamp}(\bar{m}, c_{\min}, 1.0)$ satisfies $\gamma_{\text{cautious}} \in [1.0, 1/c_{\min}]$.

Q4. Half-Precision Safe-Zone & Subnormal Numerical Floor: For half-precision execution (FP16 with in-container Bitwise-BF16 state promotion and unified $\epsilon_{\text{eff}} = \epsilon = 10^{-8} > 0$), prove that parameter update safety bounds remain strictly above the IEEE 754 FP16 subnormal threshold ($\text{floor}_{\text{sub}} = 6.1035 \times 10^{-5}$): $b_{\text{safe}} = \max(0.01|\theta_0|, 1.5 \times 10^{-4}) \geq 1.5 \times 10^{-4} > \text{floor}_{\text{sub}}$, while FP16 variance clamping $v_{\text{fp16}} = \min(v_{\text{raw}}, 65{,}000.0)$ remains strictly below the FP16 overflow ceiling (65,504.0).

Q5. Descent Direction Guarantee: Prove that under zero initial momentum ($m_0 = 0$) or cautious alignment masking [10] ($m_{\text{mask}} = \mathbb{I}(u_t \cdot g_t > 0)$ with $\gamma_{\text{cautious}} \geq 1.0$), the effective update preserves non-negative alignment with the gradient: $u_0 \cdot g_t \geq 0$ and $\delta_{\text{cautious}} \cdot g_t \geq 0$.

Q6. Weight Decay Contraction: For base learning rate $\eta \in (0, 0.05]$ and decoupled weight decay [5] $\lambda_{\text{wd}} \in [0, 0.2]$, prove that $(1 - \eta\lambda_{\text{wd}}) \in (0, 1]$. Negated query: $(1 - \eta\lambda_{\text{wd}}) \leq$

$0 \vee (1 - \eta\lambda_{\text{wd}}) > 1$.

Q7. Lookahead Bias Correction Positivity: For all integer steps $t \geq 1$ and $\beta_1, \beta_2 \in (0, 1)$ with $0 < \beta_1^{t+1} \leq \beta_1^2$ and $0 < \beta_2^t \leq \beta_2$, prove that the Nesterov lookahead bias correction [11] $\text{bc}_{1,\text{nes}} = 1 - \beta_1^{t+1}$ and clamped second-moment correction $\text{bc}_2 = \max(1 - \beta_2^t, 10^{-15})$ remain strictly positive. Negated query: $\text{bc}_{1,\text{nes}} \leq 0 \vee \text{bc}_2 \leq 0$.

2.2 Standard Z3 SMT Verification Harness

Listing 1 provides the executable 12-query Stage I test script (`eval/modules/smt.py`) verifying Single-Buffer TD-CLM and Dual-Buffer Convex Lookahead Momentum (CLM) invariants.

Listing 1: Standard LSO-Eval Stage I.1: 12-Query Z3 SMT Verification Harness (v1.1.0)

```python
from z3 import If, Or, Real, Reals, Solver, unsat

def run_z3_smt_audit() -> bool:
    solver = Solver()
    m, g, w1, w2, lam, eps = Reals("m g w1 w2 lam eps")
    base_conds = [w1 > 0, w2 > 0, (w1 + w2) == 1, lam >= 0, eps > 0]
    abs_m, abs_g = If(m >= 0, m, -m), If(g >= 0, g, -g)
    diff = g - m
    abs_diff = If(diff >= 0, diff, -diff)
    clm_m = w1 * m + w2 * g
    clm_env = w1 * abs_m + w2 * abs_g + lam * abs_diff + eps
    abs_clm_m = If(clm_m >= 0, clm_m, -clm_m)

    # 1-2. TD-CLM Non-Singularity (Q1) & Unit Metric Bound |u| < 1.0 (Q2)
    solver.push(); solver.add(*base_conds, clm_env <= 0)
    res_tdclm_div0 = solver.check(); solver.pop()
    solver.push(); solver.add(*base_conds, abs_clm_m >= clm_env)
    res_tdclm_bound = solver.check(); solver.pop()

    # 3-4. Buffer-2 Sigma Positivity (Q1/Q2) & Cautious Scale [1, 1/c_min] (Q3)
    sqrt_v, c1, c2, m_bar, c_min = Reals("sqrt_v c1 c2 m_bar c_min")
    sigma = sqrt_v * c1 + c2
    sigma_clamped = If(sigma >= 1e-12, sigma, 1e-12)
    solver.push(); solver.add(sqrt_v >= 0, c1 > 0, c2 > 0, Or(sigma <= 0, sigma_clamped <=
    0))
    res_sigma = solver.check(); solver.pop()
    solver.push(); solver.add(c_min == 0.5, m_bar >= c_min, m_bar <= 1.0)
    solver.add(Or((1.0 / m_bar) < 1.0, (1.0 / m_bar) > (1.0 / c_min)))
    res_cautious = solver.check(); solver.pop()

    # 5-6. Coordinate Governor B > 0 & Radial Lower Bound R >= B_ratio (Q2)
    p, B_ratio, norm_p = Reals("p B_ratio norm_p")
    solver.push(); solver.add(B_ratio > 0, (B_ratio * B_ratio) * (p * p + 1e-4) <= 0)
    res_coord_b = solver.check(); solver.pop()
    solver.push(); solver.add(B_ratio > 0, norm_p >= 0)
    R = If(norm_p >= 1.0, B_ratio * norm_p, B_ratio * 1.0)
    solver.add(R < B_ratio); res_radial_r = solver.check(); solver.pop()

    # 7-8. FP16 Safe-Zone Floor >= 1.5e-4 & Subnormal/Variance Safety (Q4)
    p_fp16, v_raw = Reals("p_fp16 v_raw")
    p_scaled = 0.01 * If(p_fp16 >= 0, p_fp16, -p_fp16)
    b_safe = If(p_scaled >= 1.5e-4, p_scaled, 1.5e-4)
    v_fp16 = If(v_raw <= 65000.0, v_raw, 65000.0)
    solver.push(); solver.add(b_safe < 1.5e-4); res_fp16 = solver.check(); solver.pop()
    solver.push(); solver.add(Or(b_safe <= 6.1035e-5, v_fp16 > 65504.0))
    res_subnormal = solver.check(); solver.pop()

    # 9-12. Radial Scale (Q3), Bias Corr (Q7), Weight Decay (Q6), Descent (Q5)
    r, lr, wd = Reals("r lr wd")
    solver.push(); solver.add(r >= 0)
```

```
    s_radial = If(r == 0, 1.0, (15.0 + r * r) / (15.0 + 6.0 * r * r))
    solver.add(Or(s_radial <= 0.0, s_radial > 1.0)); res_radial_scale = solver.check();
    solver.pop()

    beta1, beta2, b1_tp1, b2_t = Reals("beta1 beta2 b1_tp1 b2_t")
    solver.push(); solver.add(beta1 > 0, beta1 < 1, beta2 > 0, beta2 < 1,
              b1_tp1 > 0, b1_tp1 <= beta1 * beta1, b2_t > 0, b2_t <= beta2)
    bc2_clamped = If(1.0 - b2_t >= 1e-15, 1.0 - b2_t, 1e-15)
    solver.add(Or((1.0 - b1_tp1) <= 0, bc2_clamped <= 0)); res_bias_corr = solver.check();
    solver.pop()

    solver.push(); solver.add(lr > 0, lr <= 0.05, wd >= 0, wd <= 0.2)
    solver.add(Or((1.0 - lr * wd) <= 0, (1.0 - lr * wd) > 1)); res_wd = solver.check();
    solver.pop()

    g_val, u_val, w2_val, lam_val, eps_val, gamma_c = Reals("g_val u_val w2_val lam_val
    eps_val gamma_c")
    solver.push(); solver.add(w2_val > 0, lam_val >= 0, eps_val > 0, gamma_c >= 1.0)
    abs_g_val = If(g_val >= 0, g_val, -g_val)
    u_zero_m = (w2_val * g_val) / (w2_val * abs_g_val + lam_val * abs_g_val + eps_val)
    delta_cautious = u_val * If((u_val * g_val) > 0, 1.0, 0.0) * gamma_c
    solver.add(Or(u_zero_m * g_val < 0, delta_cautious * g_val < 0)); res_descent =
    solver.check(); solver.pop()

    return all(s == unsat for s in [
        res_tdclm_div0, res_tdclm_bound, res_sigma, res_cautious, res_coord_b,
    res_radial_r,
        res_fp16, res_subnormal, res_radial_scale, res_bias_corr, res_wd, res_descent
    ])
```

2.3 Symbolic Calculus, Rational Padé Audit & Bitwise Precision

After passing SMT verification, the optimizer undergoes four analytical and bitwise precision checks (implemented via nine automated tests in `eval/modules/math_verification.py`):

S1. Large-Gradient Limit & Equilibrium Check: Using SymPy [12], evaluate the symbolic limit of velocity u_t as gradient magnitude approaches infinity:

$$u_\infty = \lim_{|g|\to\infty} u_t(m, g). \tag{1}$$

Acceptance Rule: For Profile A (Coordinate-Bounded), the limit must remain strictly below unit magnitude ($|u_\infty| < 1.0$) and match the closed-form spike–reversal equilibrium λ^*. For Profile B and C, the limit must remain finite without numerical overflow ($\lim_{|g|\to\infty}\|u_t\| < \infty$).

S2. Piecewise Continuity (C^0/C^1 Transition) & Derivative Bounds: When radial bounding switches from unity scaling ($S(r) = 1.0$ for $r < r_{\text{thresh}}$) to $\tanh(r)/r$ at threshold $r_{\text{thresh}} = 10^{-4}$, compute the transition step:

$$\Delta_{\text{jump}} = \left|1.0 - \frac{\tanh(r_{\text{thresh}})}{r_{\text{thresh}}}\right| \approx \frac{r_{\text{thresh}}^2}{3}. \tag{2}$$

Similarly, for coordinate bounding $g(x) = B\tanh(x/B)$, verify that $g'(0) = 1.0$ and $\lim_{x\to\infty} g'(x) = 0.0$. **Acceptance Rule:** Δ_{jump} must be strictly smaller than single-precision machine epsilon ($\epsilon_{\text{FP32}} \approx 1.19 \times 10^{-7}$). At $r_{\text{thresh}} = 10^{-4}$, $\Delta_{\text{jump}} \approx 3.33 \times 10^{-9} \ll \epsilon_{\text{FP32}}$.

S3. Rational Padé [13/6] Approximation Accuracy: For hardware backends without native single-cycle tanh instructions (x86_64 AVX-512/AVX2 and Vulkan SPIR-V), $\tanh(x)$ is evaluated via a pole-free clamped [13/6] rational Padé approximant [13] on $x \in [-7.9988117, 7.9988117]$:

$$\tanh_{\text{Padé}}(x) = \frac{x \sum_{k=0}^{6} \alpha_{2k+1} x^{2k}}{\sum_{j=0}^{3} \beta_{2j} x^{2j}}. \tag{3}$$

Acceptance Rule: Maximum absolute error across $\mathbb{R}$ against IEEE 754 `std::f32::tanh` must satisfy $\| \tanh_{\text{Padé}}(x) - \tanh(x) \|_\infty < 1.5 \times 10^{-6}$, while preserving exact odd symmetry $\tanh_{\text{Padé}}(-x) = -\tanh_{\text{Padé}}(x)$ and IEEE NaN propagation.

S4. Stochastic Rounding Unbiasedness & MurmurHash3 Determinism: Low-precision quantization (`bfloat16` and `float16`) [14, 15] uses the stateless MurmurHash3 32-bit finalizer seeded deterministically per parameter and step:

$$\text{seed}_m = (\text{seed}_0 + t \cdot \texttt{0x9E3779B9} + \text{idx}_{\text{param}} \cdot 10007) \ \&\ \texttt{0x7FFFFFFF}, \tag{4}$$

$$\text{seed}_p = (\text{seed}_m + \texttt{0x55555555}) \ \&\ \texttt{0x7FFFFFFF}. \tag{5}$$

For FP16, the bitmask width adapts to subnormal exponents ($\exp_{\text{f32}} < 113$) via $\text{shift}_{\text{total}} = 13 + \text{clamp}(113 - \exp_{\text{f32}}, 0, 11)$. Generate $N = 200{,}000$ samples across both normal ($[10^{-3}, 10^{-1}]$) and FP16 subnormal ($[10^{-5}, 6.1 \times 10^{-5}]$) ranges. Evaluate unbiasedness via a two-sided Student's t-test ($H_0 : E[\mathcal{Q}_{\text{SR}}(x)] = x$) and Two One-Sided Tests (TOST) within tolerance $\pm\delta_{\text{tol}} = 10^{-6}$. **Acceptance Rule:** Maximum mean bias must satisfy $|\mu_{\mathcal{Q}} - x| < 10^{-6}$ ($p_{\text{TOST}} < 0.01$, or t-test $p \geq 0.01$) across both normal and subnormal ranges.

Reference Case Example: LuminaV (v1.5.3) TD-CLM & Dual-Buffer CLM Verification in Stage I

In LuminaV (`v1.5.3`, uncentered defect mode), the first moment updates via $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$. Single-Buffer TD-CLM (`buffer=1`) and Dual-Buffer Convex Lookahead Momentum (`buffer=2`) compute raw update velocity via convex lookahead interpolation [11], triangle-defect norm envelopes [2], and decorrelated belief variance [6, 7]:

$$u_t^{(\text{B1})} = \frac{w_1 m_{t-1} + w_2 g_t}{w_1 |m_{t-1}| + w_2 |g_t| + \lambda |d_t^{(\text{B1})}| + \epsilon_{\text{eff}}}, \quad w_1 = \beta_1^2, \quad w_2 = 1 - \beta_1^2,$$

$$u_t^{(\text{B2})} = \tanh\left(\frac{\beta_1 m_t + (1 - \beta_1) g_t}{\max(c_1 \sqrt{\max(v_t, 0)} + c_2, 10^{-12})} \right), \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left(d_t^{(\text{B2})} \right)^2,$$

where the coordinate-wise innovation defects and lookahead bias-correction scalars are defined as:

$$d_t^{(\text{B1})} = g_t - m_{t-1}, \quad d_t^{(\text{B2})} = g_t - m_t,$$

$$c_1 = \frac{(1 - \beta_1^{t+1}) \tau}{\sqrt{\max(1 - \beta_2^t, 10^{-15})}}, \quad c_2 = \epsilon_{\text{eff}} (1 - \beta_1^{t+1}) \tau.$$

Note the algebraic identity $\beta_1 m_t + (1-\beta_1) g_t \equiv \beta_1^2 m_{t-1} + (1-\beta_1^2) g_t = w_1 m_{t-1} + w_2 g_t$, unifying the Nesterov lookahead numerator across both buffer modes.

1. **12-Query Z3 SMT Result:** By the triangle inequality [2], $|w_1 m_{t-1} + w_2 g_t| \leq w_1 |m_{t-1}| + w_2 |g_t|$. Because $\lambda |d_t^{(\mathrm{B1})}| \geq 0$ and $\epsilon_{\text{eff}} = 10^{-8} > 0$, the Buffer 1 denominator strictly exceeds the numerator, while Buffer 2 satisfies $|\tanh(\cdot)| < 1.0$ with denominator floor $10^{-12} > 0$. All 12 negated queries in Listing 1 returned `PASSED (UNSAT)`.
2. **Large-Gradient Ceiling & Bias Correction:** SymPy limit analysis on Buffer 1 yields $u_\infty = \frac{w_2}{w_2+\lambda} = \frac{0.19}{0.49} \approx 0.3878 < 1.0$ and equilibrium $\lambda^* = \frac{2(0.19)^2}{0.24} \approx 0.3008$. Buffer 2 bias correction c_1 evaluates to 4.8067 at $t = 1$ and converges monotonically to $\tau = 0.8000$ as $t \to \infty$.
3. **Padé [13/6] & Stochastic Rounding Precision:** Across 500,000 evaluation points on $[-8, 8]$, [13/6] Padé tanh achieved maximum absolute error $\mathbf{9.48 \times 10^{-8}} \ll 1.5 \times 10^{-6}$ with exact odd symmetry. Monte Carlo evaluation over 200,000 samples of MurmurHash3 bitwise rounding across normal (± 0.12345678) and FP16 subnormal ($\pm 3.0 \times 10^{-5}$) values yielded maximum absolute bias $\mathbf{3.53 \times 10^{-7}} < 10^{-6}$ ($p_{\text{TOST}} = \mathbf{0.00} < 0.01$).

3 Stage II: Synthetic Test Surfaces, Cross-Backend Parity & State Serialization

Stage II connects mathematical verification (Stage I) to neural network training. Its goal is to test optimizer trajectories on challenging synthetic surfaces, calibrate default hyperparameters, verify numerical consistency across hardware backends, and check state serialization integrity.

3.1 Synthetic Benchmark Functions

The engineering team evaluates the candidate optimizer against reference baselines (standard AdamW [5] in both FP32 Master and Native BF16 precision, plus the previous algorithm baseline) across five standard test functions [16]:

B1. Ill-Conditioned Quadratic Bowl ($\kappa \geq 1000$):

$$f(w_0, w_1) = \frac{1}{2}\left(w_0^2 + 1000 w_1^2\right), \quad w^{(0)} = [10.0, 1.0]^\top. \tag{6}$$

Evaluation Focus: Tests whether the triangle-defect envelope [2] or belief variance [6] dampens steep-wall oscillations along w_1 without slowing progress along the flat direction w_0.

B2. Non-Convex Rosenbrock Curved Valley [16, 17]:

$$f(w_0, w_1) = (1 - w_0)^2 + 100(w_1 - w_0^2)^2, \quad w^{(0)} = [-1.5, 2.0]^\top. \tag{7}$$

Evaluation Focus: Tests the optimizer's ability to follow curved non-linear valleys where the gradient direction shifts continuously.

B3. Stochastic Gradient Noise Resilience: Evaluates the quadratic bowl under additive Gaussian gradient noise $\tilde{g}_t = \nabla f(w_t) + \mathcal{N}(0, \sigma^2 I)$ with $\sigma = 1.0$. **Evaluation Focus:** Measures steady-state error and noise amplification near the optimum.

B4. Multimodal Rastrigin Local-Minima Escape [18]:

$$f(w_0, w_1) = 20 + \sum_{i=0}^{1} \left(w_i^2 - 10\cos(2\pi w_i) \right), \quad w^{(0)} = [2.5, 2.5]^\top. \tag{8}$$

Evaluation Focus: Evaluates whether lookahead momentum [11] carries the iterate out of shallow local minima toward the global minimum at $(0, 0)$.

B5. High-Dimensional Saddle Point Escape ($N \geq 100$):

$$f(w) = \frac{1}{2} \sum_{i=1}^{N} \lambda_i w_i^2 + \frac{1}{4} \sum_{i=1}^{N} w_i^4, \quad \lambda_i \in [-1.0, 1000.0]. \tag{9}$$

Evaluation Focus: Evaluates the optimizer's ability to escape high-dimensional saddle points (negative curvature eigenvalues) and verifies coordinate/radial bounding stability.

3.2 Standard 2D Simulation Harness

Listing 2 provides the deterministic Python harness for Stage II trajectory testing, implementing the TD-CLM update step with cautious rescaling [10] and coordinate bounding [4].

Listing 2: Standard LSO-Eval Stage II: Deterministic Simulation Harness (v1.1.0)

```python
import numpy as np

def run_2d_benchmark(loss_fn, grad_fn, step_fn, w_init, f_star=0.0,
                     lr=0.01, max_steps=2000, tol=1e-5, noise_std=0.0, seed=1337):
    rng = np.random.default_rng(seed)
    w = np.array(w_init, dtype=np.float64)
    state = {}
    current_loss = loss_fn(w)

    for t in range(1, max_steps + 1):
        g = grad_fn(w)
        if noise_std > 0.0:
            g = g + rng.normal(0.0, noise_std, size=g.shape)

        w, state = step_fn(w, g, state, lr, t)
        current_loss = loss_fn(w)
        curr_gap = abs(current_loss - f_star)

        if np.isnan(w).any() or np.isinf(w).any() or curr_gap > 1e8:
            return t, curr_gap, "DIVERGED"
        if curr_gap < tol:
            return t, curr_gap, "CONVERGED"

    return max_steps, abs(current_loss - f_star), "MAX_STEPS"

def luminav_tdclm_step_fn(w, g, state, lr, t,
                          beta1=0.9, lam=0.3, eps=1e-8, bound_ratio=0.05):
    if "m" not in state:
        state["m"] = np.zeros_like(w)

    m = state["m"]
    w1 = beta1 * beta1
    w2 = 1.0 - w1
```

```
    m_new = beta1 * m + (1.0 - beta1) * g
    clm_m = w1 * m + w2 * g
    diff = g - m
    clm_env = w1 * np.abs(m) + w2 * np.abs(g) + lam * np.abs(diff) + eps

    u = clm_m / clm_env
    mask = (u * g > 0.0).astype(np.float64)
    cautious_scale = 1.0 / np.clip(np.mean(mask), 0.5, 1.0)

    step_val = lr * (u * mask) * cautious_scale
    b_i = bound_ratio * np.sqrt(w * w + 1e-4)
    step_val = b_i * np.tanh(step_val / b_i)

    state["m"] = m_new
    w_next = w - step_val
    return w_next, state
```

3.3 Hyperparameter Sensitivity & Equilibrium Calibration

Optimizers often fail in practice when their performance depends on a narrow hyperparameter window. LSO-Eval requires systematic sweeps across all hyperparameters:

P1. One-at-a-Time Sweep Protocol: Fix standard defaults ($\beta_1 = 0.9, \beta_2 = 0.999, \tau = 0.8, \epsilon = 10^{-8}$) and vary the target hyperparameter across at least 5 values spanning an order of magnitude.

P2. Analytical Balance Point: Whenever a parameter controls a trade-off (such as spike damping vs. sign-reversal responsiveness), derive the analytical balance point in closed form and enforce input validation bounds in code.

P3. Parameter Stability Range: The default value must sit within a stable range where convergence speed degrades by no more than 15% under a $\pm 30\%$ perturbation ($\tau \in [0.56, 1.04]$ and $r_{\text{bound}} \in [0.035, 0.065]$).

3.4 Feature Combinations, 1-Pass EMA Parity & Cross-Backend Numerical Parity

Because production optimizer suites combine multiple algorithmic options and hardware execution backends, Stage II enforces three parity checks:

A1. Feature Combination Stability: Across all 12 factorial combinations of $\mathcal{C} = \mathcal{M}_{\text{buffer}} \times \mathcal{M}_{\text{bound}} \times \mathcal{M}_{\text{cautious}}$, enabling any subset of features must remain numerically finite and convergent. Coordinate bounding applies per-parameter adaptive clipping [4] via soft hyperbolic saturation:

$$\Delta\theta_i = B_i \tanh\left(\frac{\eta\delta_i}{B_i}\right), \quad B_i = r_{\text{bound}}\sqrt{\theta_{0,i}^2 + 10^{-4}}, \tag{10}$$

while radial bounding applies unit-wise norm scaling $S(r) = \tanh(r)/r$ for $r = \frac{\eta\|\delta\|_2}{r_{\text{bound}}\max(\|\theta_0\|_2, 1.0)}$.

A2. IO-Aware 1-Pass Fused Scratchpad EMA Parity: Following IO-aware kernel fusion principles [19, 20], in 1-Pass Fused mode (`fused_single_pass=True`, active for step $t > 1$)

each parameter maintains a 6-element `float32` scratchpad vector (5 active slots + 1 reserved ABI alignment slot):

$$\mathbf{s} = [\underbrace{\Sigma m_{\text{mask}}}_{s_0}, \underbrace{\Sigma u_{\text{eff}}^2}_{s_1}, \underbrace{\Sigma \theta_0^2}_{s_2}, \underbrace{\gamma_{\text{cautious}}}_{s_3}, \underbrace{S_{\text{bound}}}_{s_4}, \underbrace{0.0}_{s_5\ \text{(reserved)}}] \in \mathbb{R}^6. \tag{11}$$

The kernel reads s_3 and s_4 from step $t-1$ while accumulating s_0, s_1, s_2 via block-level register reductions and a single `atomic_add` per thread block in one HBM memory pass, then updates s_3, s_4 post-step via EMA with mode-specific defaults (`ema_decay="auto"` $\implies$ $\alpha_{\text{ema}} = 0.95$ for `buffer=1` and $\alpha_{\text{ema}} = 0.80$ for `buffer=2`). The convergence step count of 1-Pass Fused EMA must not degrade by more than +1.0% relative to 2-Pass Instantaneous execution.

A3. Cross-Backend Numerical Parity (`luminav-core`, `luminav-simd`, `luminav-vulkan`):

- *Parallel Chunking & Pairwise Tree Reduction:* Multithreaded Rayon execution partitions tensors into chunks of `ELEM_CHUNK_SIZE` = 2,097,152 elements using 16-lane pairwise tree reduction (`reduce_sum_16`). It must achieve exact bitwise parity (`atol` = 0.0) against single-threaded scalar execution.
- *Bitwise PRNG & Stochastic Rounding Parity:* AVX-512 and AVX2 vectorized PRNG and FP16/BF16 stochastic rounding must achieve exact `0.0` bit-level parity against the scalar reference implementation.
- *Multi-Step Trajectory Parity (N = 131,072, 5 steps):* Across both TD-CLM and Dual-Buffer modes in FP32, BF16, and pure FP16 (with in-container Bitwise-BF16 state promotion), AVX-512, AVX2, and Vulkan 1.3/1.4 Wave32 UMA compute shaders must match the Scalar Reference within atol $\leq 10^{-5}$ (FP32) and atol $\leq 2 \times 10^{-2}$ (BF16/FP16).
- *Frontend Wrapper Parity:* Standalone NumPy (`StandaloneOptimizer` via zero-copy `__array_interface__`) and PyTorch (`torch.optim.Optimizer`) must produce identical parameter trajectories on matching inputs.

3.5 State Cleanup & Checkpoint Serialization Audit

To prevent silent resume bugs or memory leaks, the optimizer must pass the **LSO-Eval Serialization Audit** (Listing 3), verifying across both `float32` and `bfloat16` models and all master-weight modes (`False`, `"semi"`, `"full"`): (1) no temporary contiguous staging buffers remain in the state dictionary, (2) temporary reduction accumulators and reserved slots (s_0, s_1, s_2, s_5) are reset to zero while EMA scales (s_3, s_4) remain in `float32`, (3) `master_param` retains `float32` precision, and (4) parameter updates after reloading match an uninterrupted run bit-for-bit.

Listing 3: Standard LSO-Eval Stage II: Checkpoint Roundtrip & State Cleanup Test (v1.1.0)

```
import copy, io, torch, torch.nn as nn

def verify_checkpoint_integrity(optimizer_cls):
    torch.manual_seed(42)
    forbidden_keys = (
        "p_contig", "exp_avg_contig", "exp_avg_sq_contig",
        "mp_contig", "_plan", "_contig_step"
    )
    for dtype in (torch.float32, torch.bfloat16):
        for buf in (1, 2):
```

```
            for mw in (False, "semi", "full"):
                model_cont = nn.Sequential(nn.Linear(10, 20), nn.ReLU(), nn.Linear(20, 2)).to(dtype=dtype)
                model_res = copy.deepcopy(model_cont)
                opt_cont = optimizer_cls(model_cont.parameters(), lr=1e-3, buffer=buf, bound=True, master_weights=mw)
                opt_res = optimizer_cls(model_res.parameters(), lr=1e-3, buffer=buf, bound=True, master_weights=mw)
                batches = [(torch.randn(4, 10, dtype=dtype), torch.tensor([0, 1, 0, 1])) for _ in range(6)]

                for i in range(5):
                    x, y = batches[i]
                    for m, opt in ((model_cont, opt_cont), (model_res, opt_res)):
                        loss = nn.CrossEntropyLoss()(m(x).float(), y)
                        opt.zero_grad(); loss.backward(); opt.step()

                buffer = io.BytesIO()
                torch.save(opt_res.state_dict(), buffer); buffer.seek(0)
                opt_loaded = optimizer_cls(model_res.parameters(), lr=1e-3, buffer=buf, bound=True, master_weights=mw)
                state_dict = torch.load(buffer, weights_only=False)
                opt_loaded.load_state_dict(state_dict)

                for p_state in state_dict["state"].values():
                    for k in forbidden_keys:
                        assert k not in p_state, f"Memory leak: '{k}' in state_dict!"
                    if "scratch" in p_state:
                        sc = p_state["scratch"]
                        assert float(sc[0]) == 0.0 and float(sc[1]) == 0.0
                        assert float(sc[2]) == 0.0 and float(sc[5]) == 0.0

                for p in model_res.parameters():
                    st = opt_loaded.state[p]
                    if "scratch" in st: assert st["scratch"].dtype == torch.float32
                    if "master_param" in st: assert st["master_param"].dtype == torch.float32

                x_last, y_last = batches[5]
                for m, opt in ((model_cont, opt_cont), (model_res, opt_loaded)):
                    loss = nn.CrossEntropyLoss()(m(x_last).float(), y_last)
                    opt.zero_grad(); loss.backward(); opt.step()

                for p_c, p_r in zip(model_cont.parameters(), model_res.parameters()):
                    torch.testing.assert_close(p_c, p_r, rtol=0.0, atol=0.0)
    return True
```

Reference Case Example: LuminaV (v1.5.3) Trajectory, Equilibrium & Parity in Stage II

Proposition 1 (Symmetric Spike–Reversal Balance for λ)**.** *Let $w_1 = \beta_1^2$ and $w_2 = 1 - \beta_1^2$ with $\beta_1 \in (\frac{\sqrt{3}}{2}, 1)$. In TD-CLM [2], the velocity ceiling under a large gradient spike ($|g_t| \to \infty$) is $u_{spike}(\lambda) = \frac{w_2}{w_2+\lambda}$, and the instantaneous velocity ceiling under an exact gradient sign reversal ($g_t = -m_{t-1}$) is $u_{rev}(\lambda) = \frac{w_1-w_2}{1+2\lambda}$. The unique coefficient λ^* that equalizes spike damping and reversal damping ($u_{spike}(\lambda^*) = u_{rev}(\lambda^*)$) is given in closed form by:*

$$\lambda^* = \frac{2w_2^2}{w_1 - 3w_2} = \frac{2(1-\beta_1^2)^2}{4\beta_1^2 - 3}, \quad \forall \beta_1 \in \left(\frac{\sqrt{3}}{2}, 1\right). \tag{12}$$

Proof. Setting $\frac{w_2}{w_2+\lambda} = \frac{w_1-w_2}{1+2\lambda}$ yields $w_2(1+2\lambda) = (w_1 - w_2)(w_2 + \lambda)$. Expanding and collecting terms in λ gives $w_2(1 - w_1 + w_2) = (w_1 - 3w_2)\lambda$. Substituting $1 - w_1 = w_2$

produces $2w_2^2 = (w_1 - 3w_2)\lambda$, which solves directly to $\lambda^* = \frac{2(1-\beta_1^2)^2}{4\beta_1^2-3}$ for $4\beta_1^2 - 3 > 0 \iff \beta_1 > \frac{\sqrt{3}}{2} \approx 0.866$. Substituting standard $\beta_1 = 0.9$ yields $\lambda^* \approx 0.3008 \approx 0.30$. □

LuminaV (`v1.5.3`) was verified across all Stage II checks:

- **Synthetic Surfaces (`B1`–`B5`):**
  - *B1 Quadratic ($\kappa = 1000$, $\eta = 0.02$, 1,000 steps):* Unregularized B-CLM ($\lambda = 0.0$) required 950 steps, whereas **LuminaV Buffer 1 (TD-CLM)** converged in **575 steps** (loss gap 2.67×10^{-6}) and **LuminaV Buffer 2 (Dual)** in **571 steps** (9.31×10^{-6}). Both `AdamW_FP32_Master` (gap 3.66×10^{-2}) and `AdamW_Native_BF16` (stalled at gap 50.00) failed to reach 10^{-5}.
  - *B2 Rosenbrock ($\eta = 0.002$, 2,000 steps) & B5 Saddle ($N = 100$, $\eta = 0.02$):* On Rosenbrock, Buffer 1 reached a loss gap of 7.46×10^{-4} and Buffer 2 reached 1.76 vs. 3.90 for `AdamW_FP32_Master` and 12.50 for `AdamW_Native_BF16`. On $N = 100$ Saddle Escape, **LuminaV Buffer 2** converged fastest in **149 steps** (5.25×10^{-6}) vs. 193 steps for `AdamW_FP32_Master` and 196 steps for `AdamW_Native_BF16`.
  - *B3 Gaussian Noise ($\sigma = 1.0$) & B4 Rastrigin:* Under heavy gradient noise, Buffer 2 (8.11×10^{-2}) and Buffer 1 (2.21×10^{-1}) achieved 15×–41× lower steady-state loss than `AdamW_FP32_Master` (3.36) and `AdamW_Native_BF16` (50.00). On Rastrigin, Buffer 2 (7.9597) and Buffer 1 (7.9690) matched or beat `AdamW` (7.9597 / 7.9617).
- **Equilibrium $\lambda^* = 0.3008$ & ±30% Basin Stability:** At $\lambda = 0.30$, spike ceiling (0.3878) and reversal ceiling (0.3875) equalized, converging in 575 steps (while resonance $w_2 = \lambda = 0.19$ stalled). Perturbing $\tau \in [0.56, 1.04]$ (ref 556 steps), coordinate bound $r_{\text{bound}} \in [0.035, 0.065]$ (ref 587 steps), and radial bound $r_{\text{bound}} \in [0.035, 0.065]$ (ref 560 steps) all passed within the $\leq 15\%$ stability window.
- **1-Pass Fused Scratchpad EMA Parity:** On $\kappa = 1000$, Buffer 1 1-Pass Fused EMA ($\alpha_{\text{ema}} = 0.95$) converged in **346 steps** vs. 381 steps for 2-Pass Instantaneous ($-\mathbf{9.19}\%$ step reduction, loss 2.52×10^{-5}), and Buffer 2 1-Pass Fused EMA ($\alpha_{\text{ema}} = 0.80$) converged in **367 steps** vs. 370 steps ($-\mathbf{0.81}\%$ step reduction, loss 9.24×10^{-5}).
- **Cross-Backend & Serialization Parity:** All 12 factorial ablation combinations and all 24 checkpoint roundtrip configurations ($\{\text{FP32}, \text{BF16}\} \times \{\text{Buf 1}, \text{Buf 2}\} \times \{\text{False}, \text{semi}, \text{full}\}$) passed with zero staging leaks and bit-exact resumed trajectories (`atol` $= 0.0$).

4 Stage III: Short-Run Proxy Model Screening

Stage III tests candidate configurations on smaller proxy neural networks before committing to full training runs. By running short training schedules on proxy models, teams filter out unstable settings and verify low-precision training dynamics early.

4.1 Proxy Model Selection & Step Budget

The engineering team selects a smaller proxy model representative of the target architecture (e.g., a multi-layer causal Transformer with Rotary Position Embeddings (RoPE), SwiGLU MLP, and RMSNorm):

M1. Step Limit: The screening run is capped at 5% to 15% of the full training budget (typically 500 to 2,000 optimization steps).

M2. Batch Size Diversity: Evaluate at least two effective batch sizes (standard $1\times$ micro-batch and $4\times$ gradient-accumulated batch) to check update stability under different gradient noise levels.

M3. Controlled Schedule: Use an identical linear warmup (5% of steps) followed by cosine decay to 5% of peak learning rate across all candidates.

4.2 Early Stopping & Rejection Criteria

A candidate run is stopped and rejected immediately if any of the following three failure modes occur:

F1. Update-to-Weight Ratio Breach: Compute the layer-wise relative update norm across weight matrices (ndim ≥ 2, such as attention projections `q_proj` and MLP gates `gate_proj`, excluding 1D biases and normalization scales):

$$R_{t,l} = \frac{\|\delta_{t,l}\|_2}{\|\theta_{t,l}\|_2}. \tag{13}$$

Rejection Rule: If $R_{t,l} > 0.10$ for any matrix layer l after warmup (excessive step size) or $R_{t,l} < 10^{-7}$ during active training ($t > t_{\text{warmup}}$ and $\eta_t \geq 0.1\eta_{\max}$, indicating stalled updates), the configuration is rejected.

F2. Unrecovered Loss Spikes: If the exponential moving average (EMA, $\alpha = 0.99$) training loss jumps by more than 0.5 nats above its pre-spike minimum and fails to recover within 50 consecutive steps, the run is rejected.

F3. Low-Precision Stalling (Sub-ULP Update Cancellation): Compare the final EMA training loss of the native low-precision (`bfloat16` or `float16`) proxy trial against an identical reference trial executed in full `float32` precision [14, 15]:

$$\Delta\mathcal{L}_{\text{stall}} = \mathcal{L}_{\text{EMA}}^{(\text{BF16/FP16})}(T) - \mathcal{L}_{\text{EMA}}^{(\text{FP32-Ref})}(T). \tag{14}$$

Rejection Rule: If $\Delta\mathcal{L}_{\text{stall}} \geq 0.25$ nats, the update rule suffers from low-precision mantissa truncation and is rejected.

4.3 Learning Rate & Bounding Mode Grid

Because bounded update velocities interact with the base learning rate, engineers evaluate a learning rate grid ($\eta \in [8\times10^{-4}, 4\times10^{-2}]$) paired with bounding modes (`bound_type="coordinate"` vs. `"radial"`, default `bound_ratio=0.05`):

- **Long-Run Setting ($>$ 20k steps):** Identify conservative learning rates ($\eta \in [3\times10^{-4}, 8\times10^{-4}]$, $\lambda_{\text{wd}} = 8\times10^{-2}$) where single-buffer and dual-buffer modes maintain stable update ratios.
- **Short-Run Setting ($<$ 10k steps):** Identify higher learning rates ($\eta = 10^{-2}$) kept stable by coordinate-wise hyperbolic bounding [4].

Reference Case Example: LuminaV (v1.5.3) Proxy Model Screening in Stage III

Stage III screening was executed on `LuminaV v1.5.3` using `MProxyLLM` (4-layer causal Transformer with RoPE, SwiGLU, and RMSNorm; $V = 2{,}048, d_{\text{model}} = 256, n_{\text{head}} = 4, d_{\text{ffn}} = 682$; 500 steps, 25-step warmup, batch size 8, sequence length 128) across 1× and 4× gradient accumulation:

Configuration	BF16 EMA	FP32 Ref	Max $R_{t,l}$	Min $R_{t,l}$	F_1	F_2	F_3 (Verdict)
LuminaV Buf 1 (1×, **LR** 8×10^{-4})	0.9018	0.9021	0.01574	1.24×10^{-3}	PASS	PASS	PASS (**QUALIFIED**)
LuminaV Buf 1 (4×, **LR** 8×10^{-4})	**0.8462**	**0.8462**	0.01602	1.24×10^{-3}	PASS	PASS	PASS (**QUALIFIED**)
LuminaV Buf 2 (1×, **LR** 8×10^{-4})	0.9146	0.9141	0.01604	1.03×10^{-3}	PASS	PASS	PASS (**QUALIFIED**)
LuminaV Buf 2 (4×, **LR** 8×10^{-4})	0.8518	0.8515	0.01684	9.81×10^{-4}	PASS	PASS	PASS (**QUALIFIED**)
LuminaV Buf 1 (1×, **LR** 1×10^{-2})	0.9326	0.9317	0.04041	3.03×10^{-3}	PASS	PASS	PASS (**QUALIFIED**)
AdamW Native BF16 (1×, LR 8×10^{-4})	0.9763	0.9552	0.01423	7.76×10^{-5}	PASS	PASS	PASS (QUALIFIED)
AdamW Native BF16 (4×, LR 8×10^{-4})	0.8767	0.8678	0.01600	7.50×10^{-5}	PASS	PASS	PASS (QUALIFIED)
AdamW FP32 Master (1×, LR 3×10^{-4})	1.0989	1.0992	0.00611	1.00×10^{-4}	PASS	PASS	PASS (QUALIFIED)
AdamW FP32 Master (4×, LR 3×10^{-4})	0.9693	0.9692	0.00708	1.00×10^{-4}	PASS	PASS	PASS (QUALIFIED)

Key Findings: All five LuminaV configurations achieved $|\mathcal{L}_{\text{BF16}} - \mathcal{L}_{\text{FP32}}| \leq 0.0009$ nats (zero BF16 precision drift without master weights), compared to a +0.0211 nat BF16 penalty on Native BF16 AdamW. Even at an aggressive learning rate of $\eta = 10^{-2}$, coordinate bounding (`bound_ratio=0.05`) kept $\max R_{t,l} = 0.04041 \leq 0.10$.

5 Stage IV: Large-Scale Pre-Training Evaluation

Stage IV is the final empirical validation phase, divided into two testing tiers:

1. **Stage IV-A (Hardware Qualification, Auto-Router & Burn-in Check):** Medium-length runs (2,500 to 5,000 steps, context length 512) with gradient accumulation (8×, 4,096 tokens/step) measuring active & isolated VRAM usage, kernel latency, hardware auto-router efficiency (`GPU_DISPATCH_THRESHOLD` = 16,384 elements), FSDP collective sharding parity, and held-out validation perplexity.
2. **Stage IV-B (Full-Schedule & Long-Context Qualification):** Full training runs ($\geq$ 20,000 steps, context length $\geq$ 2,048) checking long-horizon learning rate decay stability, gradient spike resilience, and held-out validation perplexity across the full schedule.

5.1 Data Pipeline Hygiene & Cryptographic Zero-Leakage Protocol

To prevent train–test overlap when streaming from massive web and bilingual corpora, LSO-Eval Stage IV enforces three data hygiene invariants prior to training:

D1. Cryptographic Hash Locking (XXH3-64 WAL): All documents used to construct the held-out validation set ($N_{\text{eval}} \geq 450{,}000$ tokens) must be hashed via 64-bit XXH3 and committed to a persistent Write-Ahead Log (`.dedup` WAL).

D2. Cross-Context Multiset Identity: When evaluating across multiple sequence lengths (e.g., $880 \times 512 = 450{,}560$ tokens in Stage IV-A and $220 \times 2{,}048 = 450{,}560$ tokens in Stage IV-B), the flattened sorted token multisets must be 100.00% bit-identical.

D3. Strict Stream Offset & WAL Inheritance: Training prefetchers must skip all raw stream records consumed during validation construction (`skip_raw_items`) and preload the validation WAL into the live `StreamDeduplicator` so 100.00% of validation hashes remain blocked throughout Stage IV-A and IV-B training.

5.2 Hardware Profiling, Isolated Optimizer Footprint & Distributed Sharding

All candidate and baseline optimizers run on identical hardware nodes with matched random seeds, data loaders, and sequence lengths. Under PyTorch Fully Sharded Data Parallel (FSDP) with `fsdp_sync=True`, local reduction values $[\Sigma m_{\text{mask}}, \Sigma u_{\text{eff}}^2, \Sigma \theta_0^2, N_{\text{local}}]$ are packed into a single asynchronous `all_reduce` call per parameter group.

Following memory-efficient optimizer literature (8-bit Optimizers [21] and GaLore [9]), Stage IV decomposes peak active PyTorch tensor memory (`torch.cuda.max_memory_allocated()`) into the fixed non-optimizer baseline $\text{VRAM}_{\text{base}}$ and the **Isolated Optimizer Footprint** VRAM_{opt} (Table 2):

$$\text{VRAM}_{\text{peak}} = \underbrace{W_{\text{model}} + G_{\text{grad}} + A_{\text{ctx}}}_{\text{Fixed PyTorch Baseline } (\text{VRAM}_{\text{base}})} + \underbrace{\text{VRAM}_{\text{opt}}}_{\text{Isolated Optimizer State \& Master Copy}} . \tag{15}$$

Note that `max_memory_allocated()` measures active tensor bytes managed by PyTorch's caching allocator; it excludes unoccupied cached blocks (`max_memory_reserved()`) as well as CUDA driver contexts, cuBLAS workspaces, and Triton module binaries ($\approx$ 1.5–3.5 GB on `nvidia-smi`). Because $\text{VRAM}_{\text{base}}$ (model weights W_{model}, gradients G_{grad}, and autograd activations A_{ctx}) is fixed by the model architecture and sequence length, isolating $\text{VRAM}_{\text{opt}} = \text{VRAM}_{\text{peak}} - \text{VRAM}_{\text{base}}$ measures the true memory efficiency of the optimizer itself [21, 9].

Table 2: Stage IV: Standard Evaluation Metrics

Dimension	Metric	Measurement Method & Tooling
1. Convergence	Raw Loss & EMA Loss	Recorded at every step; smoothed via EMA ($\alpha \in \{0.99, 0.995\}$).
2. Generalization	Held-Out Validation PPL	Evaluated on $\geq$ 450k zero-leakage validation tokens ($\exp(\mathcal{L}_{\text{val}})$).
3. Total & Opt Memory	$\text{VRAM}_{\text{peak}}$ & VRAM_{opt} (GB)	Total active tensor peak via `max_memory_allocated()` [22] and isolated $\text{VRAM}_{\text{opt}} = \text{VRAM}_{\text{peak}} - \text{VRAM}_{\text{base}}$ [21, 9].
4. Latency	Step & Opt Kernel Time (ms)	End-to-end step time and isolated CUDA Event timer around `opt.step()`.

5.3 Two-Tiered Pareto Acceptance & Held-Out Perplexity Criteria

Stage IV evaluates checkpoints on the held-out validation set:

$$\mathrm{PPL}_{\text{held-out}} = \exp\left(\frac{1}{N}\sum_{i=1}^{N}\mathcal{L}_{\mathrm{CE}}(x_i)\right). \tag{16}$$

Because single-buffer (2 B/param) and dual-buffer (4–12 B/param) optimizers target distinct hardware operating regimes [21, 9], LSO-Eval v1.1.0 enforces a **Two-Tiered Pareto Acceptance Rule** relative to the reference Native BF16 AdamW baseline [5]:

Class I. Dual-Buffer / Full-State Optimizers (≥ 4 bytes/param, Quality-Dominant Tier): The candidate maintains both first-moment and second-moment buffers (m_t, v_t). It must achieve **strictly equal or lower held-out validation perplexity** than the reference AdamW baseline at matched token count ($\Delta\mathrm{PPL}_{\text{held-out}} = \mathrm{PPL}_{\text{cand}} - \mathrm{PPL}_{\text{AdamW-BF16}} \leq 0.0$) while passing $\mathrm{F}_1 \wedge \mathrm{F}_2$ stability gates with zero extra VRAM overhead ($\Delta\mathrm{VRAM}_{\text{opt}} \leq 0.0$ GB).

Class II. Single-Buffer / State-Compressed Optimizers (≤ 2 bytes/param, Memory-Pareto Tier): The candidate eliminates the second-moment variance buffer v_t entirely to operate on memory-constrained accelerators. It qualifies as **PASSED (Memory-Pareto Qualified)** if and only if it simultaneously satisfies all four conditions:

(a) *Dynamic Stability ($F_1 \wedge F_2$):* Layer-wise update ratios remain strictly within $R_{t,l} \in [10^{-7}, 0.10]$ with zero unrecovered loss spikes across the full schedule.

(b) *Monotonic Generalization:* Held-out validation perplexity $\mathrm{PPL}_{\text{held-out}}(t)$ decreases monotonically across evaluation checkpoints without late-stage divergence or rebound.

(c) *Hardware Footprint Dominance:* Reduces isolated optimizer state memory by at least 50% ($\Delta\mathrm{VRAM}_{\text{opt}} \leq -50.0\%$, or total $\Delta\mathrm{VRAM}_{\text{peak}} \leq -15.0\%$) relative to Native BF16 AdamW (and $\Delta\mathrm{VRAM}_{\text{opt}} \leq -80.0\%$ relative to FP32 Master AdamW) while achieving faster per-step kernel latency ($T_{\text{opt}} < T_{\text{AdamW-BF16}}$).

(d) *Bounded Compression Loss Tolerance:* The validation log-perplexity gap induced by omitting the second-moment buffer remains strictly within the Stage III F_3 tolerance bound:

$$\Delta\mathcal{L}_{\text{val}} = \ln\left(\frac{\mathrm{PPL}_{\text{cand}}}{\mathrm{PPL}_{\text{AdamW-BF16}}}\right) \leq 0.25 \text{ nats}. \tag{17}$$

Reference Case Example: LuminaV (v1.5.3) 4B-Parameter Hybrid LLM Qualification in Stage IV-A & IV-B

1. Model Architecture, Data Pipeline & Cryptographic Zero-Leakage Proof: The Stage IV-A and IV-B empirical benchmarks below were evaluated on **LuminaV** `v1.5.3` (`centered_defect=False`, native `torch.bfloat16`) from scratch (100% random initialization) on the hybrid **Qwen/Qwen3.5-4B-Base** architecture (32 layers: 24 GatedDeltaNet linear-attention layers with `causal_conv1d` + 8 full SDPA layers; $\approx 4.26\times10^9$ parameters, fixed model weight + gradient footprint $W_{\mathrm{BF16}}+G_{\mathrm{BF16}} = 15.88$ GB). The data pipeline streamed from a 6-way interleaved English–Indonesian corpus filtered via Rust `FusedPipeline`, `PIIMasker`, and `StreamDeduplicator`:

- *Validation Tensor Parity:* Stage IV-A ($880 \times 512 = 450{,}560$ tokens, SHA-256

`4d278cb87e4abbe1`) and Stage IV-B ($220 \times 2{,}048 = 450{,}560$ tokens, SHA-256 `108e517e1c181cb6`) consumed exactly 622 raw stream records (`raw_offset`) and verified **100.00% identical token multisets**.

- *Zero-Leakage WAL Continuity:* All 143,475 XXH3-64 validation hashes from Stage IV-A remained 100.00% locked inside the 295,605 committed hashes of the Stage IV-B WAL (`PASSED: 100% Bit-Exact Eval & Zero Leakage`).
- *Parallel Dataset Loader Sampling Note (`v1.5.3`):* In the `v1.5.3` evaluation data loader script, parallel dataset records were sampled by filtering uniqueness based on joint `(source, domain)` pairs rather than allocating unique records evenly per domain up to each domain's total target allocation. Because all candidate and baseline optimizers shared this identical deterministic stream schedule, relative cross-optimizer comparisons in the `v1.5.3` tables remain strictly controlled.

2. Stage IV-A Qualification (Context 512, Grad Accum 8×, 4,800 Steps / 19.66M Tokens): At sequence length 512, the fixed PyTorch non-optimizer baseline (weights + gradients + autograd activations) is $\text{VRAM}_{\text{base}}^{(512)} \approx 20.87$ GB. Across the 4,800-step accumulated qualification run evaluated on 450,560 held-out tokens:

Optimizer Configuration	Final EMA	Val PPL	ΔPPL	Peak VRAM	Opt VRAM (VRAM_{opt})	Max $R_{t,l}$	Step Time	Opt Time	LSO-Eval Status
LuminaV Buffer 2 (Dual, 8×10^{-4}**)**	**4.7621**	**141.96**	**−39.21**	36.75 GB	15.88 GB (0.0%)	0.0047	841.6 ms	48.37 ms	**PASSED (Class I)**
LuminaV Buffer 1 (TD-CLM, 3×10^{-4}**)**	5.2569	195.58	+14.41	**28.81** GB	**7.94** GB (**−50.0**%)	0.0120	**829.4** ms	36.77 ms	**PASSED (Class II)**
LuminaV Buffer 1 (TD-CLM, 8×10^{-4}**)**	5.2953	221.07	+39.90	**28.81** GB	**7.94** GB (**−50.0**%)	0.0191	830.7 ms	**36.46** ms	**PASSED (Class II)**
AdamW (Native BF16, 3×10^{-4}) [5]	5.4497	181.17	+0.00	36.75 GB	15.88 GB (Ref)	0.0034	835.4 ms	41.58 ms	Baseline (BF16)
AdamW (FP32 Master, 3×10^{-4}) [5]	5.0234	153.78	−27.39	75.36 GB	54.49 GB (+243%)	0.0040	919.5 ms	123.35 ms	Baseline (Master)

3. Stage IV-B Full-Schedule Qualification (Context 2,048, Grad Accum 1×, 20,000 Steps / 40.96M Tokens): At sequence length 2,048, autograd activations expand the fixed non-optimizer baseline to $\text{VRAM}_{\text{base}}^{(2048)} \approx 28.03$ GB, while isolated optimizer state allocations remain invariant at **7.94** GB (Buffer 1) and **15.86** GB (Buffer 2):

Optimizer Configuration	Final EMA	Val PPL	ΔPPL	Peak VRAM	Opt VRAM (VRAM_{opt})	Max $R_{t,l}$	Step Time	Opt Time	LSO-Eval Status
LuminaV Buffer 2 (Dual, 8×10^{-4}**)**	**4.5549**	**89.14**	**−43.43**	43.89 GB	15.86 GB (0.0%)	0.0189	297.1 ms	48.31 ms	**PASSED (Class I)**
LuminaV Buffer 1 (TD-CLM, 3×10^{-4}**)**	4.9173	165.13	+32.56	**35.97** GB	**7.94** GB (**−50.0**%)	0.0087	**283.5** ms	**36.99** ms	**PASSED (Class II)**
AdamW (Native BF16, 3×10^{-4}) [5]	4.7621	132.57	+0.00	43.89 GB	15.86 GB (Ref)	0.0037	288.0 ms	41.48 ms	Baseline (BF16)
AdamW (FP32 Master, 3×10^{-4}) [5]	4.8116	124.43	−8.14	80.11 GB	52.08 GB (+228%)	0.0045	370.1 ms	123.33 ms	Baseline (Master)

Key Qualification Findings:

1. **Class I Quality Dominance (LuminaV Buffer 2):** Without using FP32 master weights, `LuminaV Buffer 2 (Dual)` achieved the lowest validation perplexity in both Stage IV-A (**141.96**, beating Native BF16 AdamW by −39.21 PPL and FP32 Master AdamW by −11.82 PPL) and Stage IV-B (**89.14**, beating Native BF16 AdamW by **−43.43 PPL** and outperforming FP32 Master AdamW's 124.43 by **−35.29 PPL** while saving **36.22** GB of VRAM and running 2.55× faster in optimizer step latency).
2. **Class II Memory-Pareto Qualification (LuminaV Buffer 1):** `LuminaV Buffer 1 (TD-CLM, LR` 3×10^{-4}`)` reduced isolated optimizer state memory by exactly **50.0**% (7.94 GB vs. 15.88 GB on BF16 AdamW) and reduced optimizer memory overhead by **84.8**%–**85.4**% (7.94 GB vs. 52.08–54.49 GB on FP32 Master AdamW, saving 44.14–46.55 GB of total HBM so 4B pre-training fits comfortably on a single 40 GB GPU where FP32 Master AdamW fails with OOM). Its kernel latency (36.46–36.99 ms) was 3.33× faster than FP32 Master AdamW (123.33 ms), while its validation log-perplexity gap remained strictly bounded within Class II tolerances ($\Delta\mathcal{L}_{\text{val}} = 0.0765 \leq 0.25$ nats in IV-A and $0.2196 \leq 0.25$ nats in IV-B).

6 V-Eval: Vision & Spatial Modality Qualification Protocol

Stages I and II verify general mathematical and numerical properties, while Stages III and IV evaluate language models. Computer vision models behave differently during training: convolutional layers, batch and response normalization, normalization-free networks, vision transformers under strong image augmentation, and flow-matching generative models each place different demands on an optimizer. **V-Eval** is a standalone vision evaluation protocol that tests these models without changing the four-stage LLM pipeline.

6.1 Model Sources and Two-Track Structure

To keep comparisons fair and reproducible, V-Eval requires using standard, unaltered models from established public libraries—`timm` (PyTorch Image Models), `torchvision.models`, Hugging Face `transformers`, `ultralytics`, and the official `facebookresearch/SiT` repository—rather than writing custom vision backbones.

V-Eval is divided into two tracks:

1. **Track V-A (Core Mandatory — Vision From-Scratch Qualification):** Trains five standard vision architectures (`V-A1` through `V-A5`) from random initialization (`pretrained=False`, `seed=42`) in `torch.bfloat16`.
2. **Track V-B (Optional Extended Tier — Vision Downstream Fine-Tuning):** Evaluates fine-tuning of pretrained backbones on object detection (`V-B1`) and fine-grained classification (`V-B2`) when additional Compute Units (CU) are available. This track tests whether an optimizer preserves pretrained features without catastrophic forgetting or update stalling at small learning rates ($\eta \leq 10^{-4}$) in `bfloat16`.

6.2 Six-Optimizer Comparison Matrix and Parameter Grouping

To compare native `bfloat16` training (without master weights) against `float32` master-weight training across both two-buffer and single-buffer optimizers, every model in Track V-A and Track V-B is tested with the same six configurations:

1. **`LuminaV_Buffer2_Dual` (Class I Candidate — Native BF16, Master-Free):** `buffer=2`, `tau=0.8`, `betas=(0.9, 0.999)`, `eps=1e-8`, `weight_decay=0.08`, `bound=True`, `bound_type="coordinate"`, `bound_ratio=0.05`, `master_weights="none"`, `stochastic_rounding=True`, `fused_single_pass=True`, `fuse_tensors=True`, `bucket_cap_mb=500.0`, `prewarm_jit=True`.
2. **`LuminaV_Buffer1_TDCLM` (Class II Candidate — Native BF16, Master-Free):** `buffer=1`, `lam=0.3`, `betas=(0.9, 0.999)`, `eps=1e-8`, `weight_decay=0.08`, `bound=True`, `bound_type="coordinate"`, `bound_ratio=0.05`, `master_weights="none"`, `stochastic_rounding=True`, `fused_single_pass=True`, `fuse_tensors=True`, `bucket_cap_mb=500.0`, `prewarm_jit=True`.
3. **`AdamW_Native_BF16` (Baseline 2-Buffer — Native BF16, Master-Free):** `torch.optim.AdamW` [5], `betas=(0.9, 0.999)` (or `(0.9, 0.95)`), `eps=1e-8`, `weight_decay=0.08` (From-Scratch) / `0.05` (Fine-Tuning), `fused=True`, `use_fp32_master=False`.
4. **`AdamW_FP32_Master` (Reference 2-Buffer — BF16 + FP32 Master Copy):** Same hyperparameters as `AdamW_Native_BF16`, with an explicit `float32` master weight copy

(`use_fp32_master=True`).

5. **`Lion_Native_BF16` (Baseline 1-Buffer — Native BF16, Master-Free):** `SimpleLion` [3] (Listing 4), `betas=(0.95, 0.98)` (or `(0.9, 0.99)`), `weight_decay=0.01` (or LR-scaled `0.08`), `use_fp32_master=False`.
6. **`Lion_FP32_Master` (Reference 1-Buffer — BF16 + FP32 Master Copy):** Same hyperparameters as `Lion_Native_BF16`, with an explicit `float32` master weight copy (`use_fp32_master=True`).

Weight Decay Parameter Grouping (`split_vision_parameter_groups`): For all six optimizers, model parameters are split into two groups before training (Listing 4):

- **Decayed Group ($\lambda_{\text{wd}} > 0$):** All weight tensors with ndim ≥ 2, including standard and depthwise convolutions (`nn.Conv2d`), linear layers (`nn.Linear`), and attention QKV/MLP projection matrices.
- **No-Decay Group ($\lambda_{\text{wd}} = 0.0$):** All 1D parameters (ndim ≤ 1), biases, normalization parameters (`BatchNorm2d`, `LayerNorm`, `RMSNorm`, `GroupNorm`, `GlobalResponseNorm`/`GRN`), and positional/class tokens (`cls_token`, `pos_embed`, `relative_position_bias_table`).

Listing 4: Standard LSO-Eval V-Eval: Standalone SimpleLion Optimizer (Chen et al., NeurIPS 2023, Algorithm 2) & Vision Parameter Grouping

```
import torch
import torch.nn as nn

class SimpleLion(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-4, betas=(0.95, 0.98), weight_decay=0.01):
        if lr < 0.0:
            raise ValueError(f"Invalid learning rate: {lr}")
        if not 0.0 <= betas[0] < 1.0:
            raise ValueError(f"Invalid beta parameter at index 0: {betas[0]}")
        if not 0.0 <= betas[1] < 1.0:
            raise ValueError(f"Invalid beta parameter at index 1: {betas[1]}")
        if weight_decay < 0.0:
            raise ValueError(f"Invalid weight_decay value: {weight_decay}")
        defaults = dict(lr=lr, betas=betas, weight_decay=weight_decay)
        super().__init__(params, defaults)

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()
        for group in self.param_groups:
            lr, (beta1, beta2), wd = group["lr"], group["betas"], group["weight_decay"]
            for p in group["params"]:
                if p.grad is None:
                    continue
                grad = p.grad
                if grad.is_sparse:
                    raise RuntimeError("SimpleLion does not support sparse gradients.")
                state = self.state[p]
                if len(state) == 0:
                    state["exp_avg"] = torch.zeros_like(p,
    memory_format=torch.preserve_format)
                exp_avg = state["exp_avg"]
                if wd != 0.0:
                    p.mul_(1.0 - lr * wd)
                update = exp_avg.lerp(grad, 1.0 - beta1).sign_()
                p.add_(update, alpha=-lr)
```

```
                exp_avg.lerp_(grad, 1.0 - beta2)
        return loss

def split_vision_parameter_groups(model: nn.Module, weight_decay: float = 0.08):
    no_decay_kw = (
        "bias", "bn", "norm", "ln", "rmsnorm", "groupnorm", "grn",
        "cls_token", "pos_embed", "relative_position_bias_table", "gamma", "beta",
    )
    decay, no_decay = [], []
    for name, param in model.named_parameters():
        if not param.requires_grad:
            continue
        if param.ndim <= 1 or any(k in name.lower() for k in no_decay_kw):
            no_decay.append(param)
        else:
            decay.append(param)
    return [
        {"params": decay, "weight_decay": float(weight_decay)},
        {"params": no_decay, "weight_decay": 0.0},
    ]
```

6.3 Track V-A: Vision From-Scratch Qualification (Mandatory)

All Track V-A models are trained from scratch (`pretrained=False`, `seed=42`) in `torch.bfloat16` on a single GPU (reference target: 1× NVIDIA A100 40 GB). Table 3 summarizes the five test models:

V-A1. Classical CNN (`ResNet-50` [23], CIFAR 3×3 stem, $\approx$ 23.7M params): Standard residual network with `BatchNorm2d` and `ReLU` trained on CIFAR-100 (32×32, 50k train / 10k val), loaded via `timm.create_model("resnet50", pretrained=False, num_classes=100)` with a CIFAR 3×3 stem.

V-A2. Normalization-Free CNN (`NF-ResNet-50` [4], $\approx$ 25.6M params): Replaces batch normalization with Scaled Weight Standardization (`ScaledStdConv2d`), testing coordinate bounding ($\Delta\theta_i = B_i \tanh(\eta\delta_i/B_i)$) without batch normalization, loaded via `timm.create_model("nf_resnet50", pretrained=False, num_classes=100)`.

V-A3. Modern ConvNet (`ConvNeXt-V2-Tiny` [24], $\approx$ 28.6M params): Uses 7×7 depthwise convolutions, `LayerNorm`, and Global Response Normalization (`GRN`) with Mixup (0.8), CutMix (1.0), and Label Smoothing (0.1), loaded via `timm.create_model("convnextv2_tiny", pretrained=False, num_classes=100)`.

V-A4. Vision Transformer (`ViT-Small` [25], Patch 4×4, $\approx$ 22.0M params): Multi-head self-attention trained with Mixup (0.8), CutMix (1.0), RandAugment, and Label Smoothing (0.1), loaded via `timm.create_model("vit_small_patch16_224", pretrained=False, img_size=32, patch_size=4, num_classes=100)`.

V-A5. Flow-Matching Generative Model (`SiT-S/2` [27, 26], $\approx$ 33.0M params): Scalable Interpolant Transformer trained with Rectified Flow Matching velocity loss ($\mathcal{L}_{\text{FM}} = \mathbb{E}_{t,x_0,x_1}[\|v_\theta(x_t,t) - (x_1 - x_0)\|_2^2]$ and Model EMA 0.999) on CIFAR-10 (32×32), loaded from the official `SiT_models["SiT-S/2"]` implementation.

Epoch Range and Validation Schedule: Track V-A allows 50–150 epochs (9,750–29,250 steps) for CNNs (`V-A1`, `V-A2`) and 50–200 epochs (9,750–39,000 steps) for ConvNeXt, ViT, and SiT (`V-A3`–`V-A5`), with a **default target of 100 epochs (19,500 steps)** at `batch_size = 256`.

In any comparison, all six optimizers must run for the exact same number of epochs E within the permitted range, using linear warmup (5% of E for `V-A1/A2/A5`; 10% of E for `V-A3/A4`) followed by cosine decay to $\eta_{\min} = 0.05\eta$. Validation is run every 10% of total epochs (every 10 epochs at the 100-epoch default, and at the final epoch).

Table 3: Track V-A (Core Mandatory): Vision From-Scratch Qualification Matrix (`pretrained=False`, `bfloat16`)

Code	Model Backbone	Architecture Type	Dataset	Batch	Epochs (Range / Default)	Warmup & Schedule	Learning Rate (η)	Primary Metrics
V-A1	`ResNet-50` [23] (CIFAR 3×3 stem, ~ 23.7M)	Classical CNN (`BatchNorm2d` + `ReLU`)	CIFAR-100 (32×32)	256	50–150 Ep **100 Ep** (19.5k steps)	5% (5 Ep) Warmup + Cosine ($\eta_{\min} = 0.05\eta$)	LuminaV: 8e-4, 3e-4 AdamW: 1e-3 | Lion: 1e-4	Top-1/Top-5 Acc, Val Loss, $R_{t,l}$
V-A2	`NF-ResNet-50` [4] (Norm-Free, ~ 25.6M)	Normalization-Free CNN (Scaled Weight Std, No BN)	CIFAR-100 (32×32)	256	50–150 Ep **100 Ep** (19.5k steps)	5% (5 Ep) Warmup + Cosine ($\eta_{\min} = 0.05\eta$)	LuminaV: 8e-4, 3e-4 AdamW: 1e-3 | Lion: 1e-4	Top-1/Top-5 Acc, Val Loss, $R_{t,l}$
V-A3	`ConvNeXt-V2-Tiny` [24] (~ 28.6M)	Modern Pure ConvNet (DW 7×7 + `LN` + `GRN`)	CIFAR-100 (32×32)	256	50–200 Ep **100 Ep** (19.5k steps)	10% (10 Ep) Warmup + Cosine (Mixup/CutMix)	LuminaV: 8e-4, 3e-4 AdamW: 8e-4 | Lion: 1e-4	Top-1/Top-5 Acc, Val Loss, $R_{t,l}$
V-A4	`ViT-Small` [25] (Patch 4×4, ~ 22.0M)	Isotropic Vision Transformer (Multi-Head Self-Attention)	CIFAR-100 (32×32)	256	50–200 Ep **100 Ep** (19.5k steps)	10% (10 Ep) Warmup + Cosine (RandAug/Mixup)	LuminaV: 8e-4, 3e-4 AdamW: 8e-4 | Lion: 1e-4	Top-1/Top-5 Acc, Val Loss, $R_{t,l}$
V-A5	`SiT-S/2` [27] (Interpolant Transformer, ~ 33.0M)	Generative Vision (Rectified Flow Matching [26])	CIFAR-10 (32×32)	256	50–200 Ep **100 Ep** (19.5k steps)	5% (5 Ep) Warmup + Cosine (EMA 0.999)	LuminaV: 5e-4, 3e-4 AdamW: 5e-4 | Lion: 1e-4	Held-Out Velocity MSE Loss & FID

6.4 Track V-B: Downstream Fine-Tuning Qualification (Optional Tier)

Track V-B is an optional benchmark run when extra Compute Units (CU) are available. Fine-tuning starts from pretrained weights and uses small learning rates ($\eta \leq 10^{-4}$). At these step sizes, plain `bfloat16` updates without stochastic rounding can round to zero when $|\eta\delta_t| < \frac{1}{2}\mathrm{ULP}(\theta_t)$ (**Sub-ULP Update Stalling**), while poorly calibrated updates can overwrite pretrained representations (**Catastrophic Forgetting**; Table 4):

V-B1. Object Detection Fine-Tuning (`RT-DETR-R18` / `YOLOv11-S` / `FCOS-ResNet50`):

- *Initialization & Dataset:* Pretrained COCO/ImageNet backbone with a randomly initialized detection head on open object detection datasets: **`cppe-5`** (5 classes, 480×480) or **`Pascal VOC 2007`** (20 classes, 480×480).
- *Schedule & Learning Rates:* 15–35 epochs (**default 25 epochs**), `batch_size = 16` (or 32), 2-epoch linear warmup + cosine decay. Learning rates: LuminaV $\eta \in \{1 \times 10^{-4}, 2 \times 10^{-4}\}$, AdamW $\eta = 1 \times 10^{-4}$, Lion $\eta = 3 \times 10^{-5}$.
- *Loss & Metrics:* Hungarian bipartite matching / multi-task box loss (Focal + L_1 + GIoU), evaluated via **COCO mAP**$_{50:95}$, **mAP**$_{50}$, and Validation Loss every 5 epochs.

V-B2. Fine-Grained Classification Fine-Tuning (`ViT-Base/16` & `ConvNeXt-Base`):

- *Initialization & Dataset:* Pretrained ImageNet-1K backbone (`timm`) with a new linear classification head on **`Oxford-IIIT Pet`** (37 classes, $\approx$ 3,680 train / 3,669 test images at 224×224).

- *Schedule & Learning Rates:* 10–30 epochs (**default 20 epochs**), `batch_size = 64`, 2-epoch linear warmup + cosine decay. Learning rates: LuminaV $\eta = 1 \times 10^{-4}$, AdamW $\eta = 1 \times 10^{-4}$, Lion $\eta = 3 \times 10^{-5}$.
- *Metrics:* **Top-1 Accuracy (%)**, **Top-5 Accuracy (%)**, and Validation Cross-Entropy Loss evaluated every 2 epochs.

Table 4: Track V-B (Optional / Extended CU Tier): Vision Downstream Fine-Tuning Qualification Matrix

Code	Pretrained Backbone	Downstream Task	Dataset & Res.	Batch	Epochs (Range / Default)	Warmup & Schedule	Learning Rate (η)	Primary Metrics
V-B1	`RT-DETR-R18` / `YOLOv11-S` / `FCOS-R50`	End-to-End Object Detection (Hungarian / Focal + L_1 + GIoU)	`cppe-5` (5 cls) or `VOC 2007` (480 × 480)	16	15–35 Ep **25 Ep** (Eval / 5 Ep)	2 Ep Linear Warmup + Cosine Decay	LuminaV: 1e-4, 2e-4 AdamW: 1e-4 | Lion: 3e-5	COCO $\text{mAP}_{50:95}$, mAP_{50}, Val Loss
V-B2	`ViT-Base/16` [25] & `ConvNeXt-Base` [24]	Fine-Grained High-Res Transfer Classification	`Oxford-IIIT Pet` (37 cls, 224 × 224)	64	10–30 Ep **20 Ep** (Eval / 2 Ep)	2 Ep Linear Warmup + Cosine Decay	LuminaV: 1e-4 AdamW: 1e-4 | Lion: 3e-5	Top-1 & Top-5 Acc, Val CE Loss

6.5 Acceptance Criteria for Vision (V-Eval)

Following the same two-tier structure as Stage IV, V-Eval compares candidate optimizers against the native `bfloat16` baseline (`AdamW_Native_BF16`) and the `float32` master-weight reference (`AdamW_FP32_Master`):

Class I. Dual-Buffer Optimizers (`LuminaV_Buffer2_Dual`): The candidate must pass the stability checks ($\text{F}_1 \wedge \text{F}_2$, keeping $R_{t,l} \in [10^{-7}, 0.10]$ without unrecovered loss spikes), use no extra optimizer memory ($\Delta\text{VRAM}_{\text{opt}} \leq 0.0$ GB vs. `AdamW_Native_BF16`), and match or improve validation quality on the target task:

$$\Delta\text{Acc}_{\text{Top-1}} \geq 0.0\%, \qquad \Delta\text{mAP}_{50:95} \geq 0.0\%, \qquad \Delta\mathcal{L}_{\text{FM}} \leq 0.0. \tag{18}$$

Class II. Single-Buffer Optimizers (`LuminaV_Buffer1_TDCLM`): The candidate must pass the stability checks ($\text{F}_1 \wedge \text{F}_2$), cut isolated optimizer state memory by at least **50.0**% compared to `AdamW_Native_BF16` and at least **83.3**% compared to `AdamW_FP32_Master`:

$$\Delta\text{VRAM}_{\text{opt}}^{(\text{vs. BF16})} \leq -50.0\%, \qquad \Delta\text{VRAM}_{\text{opt}}^{(\text{vs. FP32-Master})} \leq -83.3\%, \tag{19}$$

run faster per optimizer step ($T_{\text{opt}} < T_{\text{AdamW-BF16}}$), and keep final validation loss within $\Delta\mathcal{L}_{\text{val}} \leq 0.25$ nats (or Top-1 accuracy within $|\Delta\text{Acc}_{\text{Top-1}}| \leq 1.5\%$) of `AdamW_Native_BF16`.

7 Differential Versioning & Regression Protocol (D-Eval)

When updating an existing optimizer or backend kernel from version v_{k-1} to v_k, LSO-Eval requires the **D-Eval Protocol** to ensure no silent performance or numerical regressions are introduced.

7.1 The Four Anti-Regression Gates

A release candidate must pass all four gates before deployment (Table 5):

Table 5: The D-Eval Four-Gate Anti-Regression Framework

Gate	Evaluation Target	Acceptance Criterion	Verification Tool
Gate 1	Algebraic Safety	All 12 Stage I SMT queries return `UNSAT`; Padé & SR bounds preserved.	Z3 [1] / SymPy [12]
Gate 2	Trajectory & Backend Parity	Head-to-head 2D run on $\kappa \geq 1000$ and cross-backend parity tests pass.	2D Sim & Rust Tests
Gate 3	Checkpoint Compatibility	Checkpoints saved in version v_{k-1} load cleanly into v_k with identical dtypes and trajectories.	State-Dict Audit
Gate 4	Production Metrics Delta	Meets non-regression bounds: $\Delta\mathcal{L} \leq 0$, $\Delta\text{VRAM}_{\text{opt}} \leq 0$, $\Delta T_{\text{kernel}} \leq 0$ (optional when update trajectory is invariant).	Stage IV (Optional)

7.2 Performance Delta Comparison

For Gate 4, the engineering team reports the performance differences against the previous version:

$$\Delta\mathcal{L}_{\text{EMA}} = \mathcal{L}_{\text{EMA}}(v_k) - \mathcal{L}_{\text{EMA}}(v_{k-1}) \leq 0, \tag{20}$$

$$\Delta\text{VRAM}_{\text{opt}} = \text{VRAM}_{\text{opt}}(v_k) - \text{VRAM}_{\text{opt}}(v_{k-1}) \leq 0, \tag{21}$$

$$\Delta T_{\text{kernel}} = T_{\text{kernel}}(v_k) - T_{\text{kernel}}(v_{k-1}) \leq 0. \tag{22}$$

If any metric regresses ($\Delta > 0$), the team must document the trade-off in an Architecture Decision Record (ADR) explaining why the change is justified.

Reference Case Example: D-Eval Regression Example: LuminaV Release Series

Summary of the D-Eval regression checks across the LuminaV release series:

- **Gate 1 (SMT & Unified ϵ_{eff}):** Unified Convex Lookahead Momentum weights ($w_1 = \beta_1^2, w_2 = 1 - \beta_1^2$) across Buffer 1 and Buffer 2 ($w_1 + w_2 = 1$), verified [13/6] Padé tanh accuracy ($9.48 \times 10^{-8} < 1.5 \times 10^{-6}$), and in `v1.5.4` promoted pure `FP16` 16-bit state containers to in-container Bitwise-BF16 (8-bit exponent range 1.18×10^{-38} at zero extra VRAM), unifying $\epsilon_{\text{eff}} = \epsilon = 10^{-8}$ across all precisions.
- **Gate 2 (Uncentered Defect Simplification & Backend Parity):** Removed the global defect mean reduction (μ_d) in `v1.5.4` to standardize on the local coordinate-wise innovation defect $d_t^{(\text{B1})} = g_t - m_{t-1}$ and $d_t^{(\text{B2})} = g_t - m_t$, eliminating pre-step reduction kernels while preserving exact trajectory parity with `v1.5.3` (`centered_defect=False`) across Scalar, AVX2, AVX-512, Vulkan 1.3/1.4 Wave32 UMA, and Triton AOT/JIT.

- **Gate 3 (Contiguous Bucket Fusion & Checkpoint Cleanliness):** Added opt-in bounded contiguous parameter coalescing (`fuse_tensors=False`, `bucket_cap_mb=500.0`) and zero-stall Triton JIT pre-warmup (`prewarm_jit=True`) in `v1.5.5`, while ensuring the 6-slot `float32` scratchpad accumulators (s_0, s_1, s_2, s_5) are zeroed on export without leaking staging keys.
- **Gate 4 (Zero-Overhead Hardware Footprint):** Verified in `v1.5.3` Stage IV qualification with $\Delta\text{VRAM}_{\text{opt}} = 0$ GB (7.94 GB isolated optimizer state on 4B LLM for Buffer 1; 15.86 GB for Buffer 2) and packaged zero-dependency sub-1MB CPython Stable ABI (`abi3-py310` / `abi3t`) wheels.

References

[1] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Budapest, Hungary, 2008, pp. 337–340.

[2] L. Maligranda, "Some remarks on the triangle inequality for norms," *Banach J. Math. Anal.*, vol. 2, no. 2, pp. 31–41, 2008.

[3] X. Chen *et al.*, "Symbolic discovery of optimization algorithms," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 49205–49233.

[4] A. Brock, S. De, S. L. Smith, and K. Simonyan, "High-performance large-scale image recognition without normalization," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, vol. 139, 2021, pp. 1059–1071.

[5] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, New Orleans, LA, USA, 2019.

[6] J. Zhuang, T. Tang, Y. Ding, S. Tatikonda, N. Dvornek, X. Papademetris, and J. S. Duncan, "AdaBelief optimizer: Adapting stepsizes by the belief in observed gradients," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 18795–18806.

[7] S. Taniguchi, K. Harada, G. Minegishi, Y. Oshima, S. C. Jeong, G. Nagahara, T. Iiyama, M. Suzuki, Yusuke Iwasawa, and Y. Matsuo, "ADOPT: Modified Adam can converge with any β_2 with the optimal rate," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024.

[8] V. Gupta, T. Koren, and Y. Singer, "Shampoo: Preconditioned stochastic tensor optimization," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, vol. 80, Stockholm, Sweden, 2018, pp. 1842–1850.

[9] J. Zhao, Z. Zhang, B. Chen, Z. Wang, A. Anandkumar, and Y. Tian, "GaLore: Memory-efficient LLM training by gradient low-rank projection," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, vol. 235, Vienna, Austria, 2024.

[10] K. Liang, L. Chen, B. Liu, and Q. Liu, "Cautious optimizers: Improving training with one line of code," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.

[11] M. R. Zhang, J. Lucas, G. E. Hinton, and J. Ba, "Lookahead optimizer: k steps forward, 1 step back," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, Vancouver, BC, Canada, 2019, pp. 9593–9604.

[12] A. Meurer *et al.*, "SymPy: Symbolic computing in Python," *PeerJ Comput. Sci.*, vol. 3, p. e103, Jan. 2017, doi: 10.1717/peerj-cs.103.

[13] A. Molina, P. Schramowski, and K. Kersting, "Padé activation units: End-to-end learning of flexible activation functions in deep networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.

[14] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *Proc. 32nd Int. Conf. Mach. Learn. (ICML)*, Lille, France, 2015, pp. 1737–1746.

[15] P. Zamirai, J. Zhang, C. R. Aberger, and C. De Sa, "Revisiting BFloat16 training," *arXiv preprint arXiv:2010.06192v2*, Mar. 2021.

[16] M. Jamil and X.-S. Yang, "A literature survey of benchmark functions for global optimization problems," *Int. J. Math. Model. Numer. Optim.*, vol. 4, no. 2, pp. 150–194, 2013, doi: 10.1504/IJMMNO.2013.055204.

[17] H. H. Rosenbrock, "An automatic method for finding the greatest or least value of a function," *Comput. J.*, vol. 3, no. 3, pp. 175–184, Oct. 1960, doi: 10.1093/comjnl/3.3.175.

[18] L. A. Rastrigin, *Systems of Extremal Control*. Moscow, Russia: Nauka, 1974.

[19] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, New Orleans, LA, USA, 2022, pp. 16344–16359.

[20] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proc. 3rd ACM SIGPLAN Int. Workshop Mach. Learn. Program. Lang. (MAPL)*, Phoenix, AZ, USA, Jun. 2019, pp. 10–19, doi: 10.1145/3315508.3329973.

[21] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer, "8-bit optimizers via block-wise quantization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[22] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, Vancouver, BC, Canada, 2019, pp. 8024–8035.

[23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.

[24] S. Woo *et al.*, "ConvNeXt V2: Co-designing and scaling ConvNets with masked autoencoders," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Vancouver, BC, Canada, 2023, pp. 16133–16142.

[25] A. Dosovitskiy *et al.*, "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[26] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Kigali, Rwanda, 2023.

[27] N. Ma, M. Goldstein, M. S. Albergo, N. M. Boffi, E. Vanden-Eijnden, and S. Xie, "SiT: Exploring flow and diffusion-based generative models with scalable interpolant transformers," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Milan, Italy, 2024.

Lumina Moon Standard Optimizer Evaluation Methodology (LSO-Eval v1.1.0)

Official Engineering Specification — Lumina Moon (cloverx-id)